



PROCESO DE DESARROLLO DE SOFTWARE

Grupo 13 **Trabajo Práctico Obligatorio**

Integrantes:

Tomas Szkarlatiuk, 1190913

Delia Tomas, 1188972

Rocha Germán, 1195326

Profesora:

LEON MARIA ANGELA

1. Introducción	3
2. Resumen de la solución de diseño	3
3. Diagrama de clases UML	4
4. Diagramas de secuencia	5
4.1. CU3 — Crear reserva	5
4.2. CU5 — Confirmar reserva	5
5. Patrones de diseño aplicados y justificación	7
5.1. Factory Method (creacional)	7
5.2. Singleton (creacional)	7
5.3. Decorator (estructural)	7
5.4. Facade (estructural)	8
5.5. State (comportamiento)	8
5.6. Strategy (comportamiento)	8
6. Principios SOLID aplicados	9
7. Patrones GRASP aplicados	9
11. Informe de avance individual por integrante	10
12. Conclusión	11

1. Introducción

El presente documento corresponde a la Fase 2 del Trabajo Práctico Obligatorio de la asignatura Proceso de Desarrollo de Software. Su objetivo es avanzar desde el análisis inicial elaborado en la Fase 1 hacia el diseño orientado a objetos de la solución, definiendo las responsabilidades de cada clase y seleccionando de manera justificada los patrones de diseño que serán aplicados.

El sistema desarrollado es un Sistema de Gestión Hotelera y Servicios para Huéspedes, pensado para que una cadena hotelera pueda centralizar y modernizar la administración de reservas, habitaciones, huéspedes, servicios adicionales, estados de estadia y promociones, manteniendo una arquitectura flexible, reutilizable y extensible con persistencia de datos.

En esta fase se presentan: el diagrama de clases UML refinado, dos diagramas de secuencia de casos de uso relevantes, la identificación y justificación de los patrones de diseño (creacionales, estructurales y de comportamiento), los principios SOLID y los patrones GRASP aplicados, la estructura de paquetes del proyecto Java y una primera versión del código con las clases principales, interfaces y relaciones, totalmente compilable y ejecutable por consola.

Trazabilidad con la Fase 1: el diseño mantiene coherencia con el análisis previo. Las clases de dominio, los actores, los casos de uso y los patrones anticipados en la Fase 1 se conservan y se concretan aquí en un modelo de diseño y en código funcional.

2. Resumen de la solución de diseño

La solución aplica seis patrones de diseño que cubren las tres categorías exigidas por la consigna (creacional, estructural y de comportamiento), además de tres principios SOLID y tres patrones GRASP. El siguiente cuadro sintetiza las decisiones de diseño antes de detallarlas:

Patrón	Categoría	Dónde se aplica	Problema que resuelve
Factory Method	Creacional	factory.HabitacionFactory	Crear habitaciones por tipo (Standard/Premium/Suite) sin acoplar el controlador a clases concretas.
Singleton	Creacional	repository.DatabaseConnection	Garantizar una única instancia de conexión a la base de datos.
Decorator	Estructural	model.servicio.*	Agregar servicios (desayuno, spa, cochera) de forma dinámica sin modificar la reserva.
Facade	Estructural	facade.SistemaHotelFacade	Ofrecer una interfaz única y simple sobre los subsistemas.

Patrón	Categoría	Dónde se aplica	Problema que resuelve
State	Comportamiento	model.state.*	Administrar los estados de la reserva y sus transiciones válidas.
Strategy	Comportamiento	strategy.*	Intercambiar algoritmos de descuento sin tocar la lógica de la reserva.

- Principios SOLID aplicados: SRP (Responsabilidad Única), OCP (Abierto/Cerrado) y DIP (Inversión de Dependencias).
- Patrones GRASP aplicados: Experto en Información, Creador, Controlador y Bajo Acoplamiento.

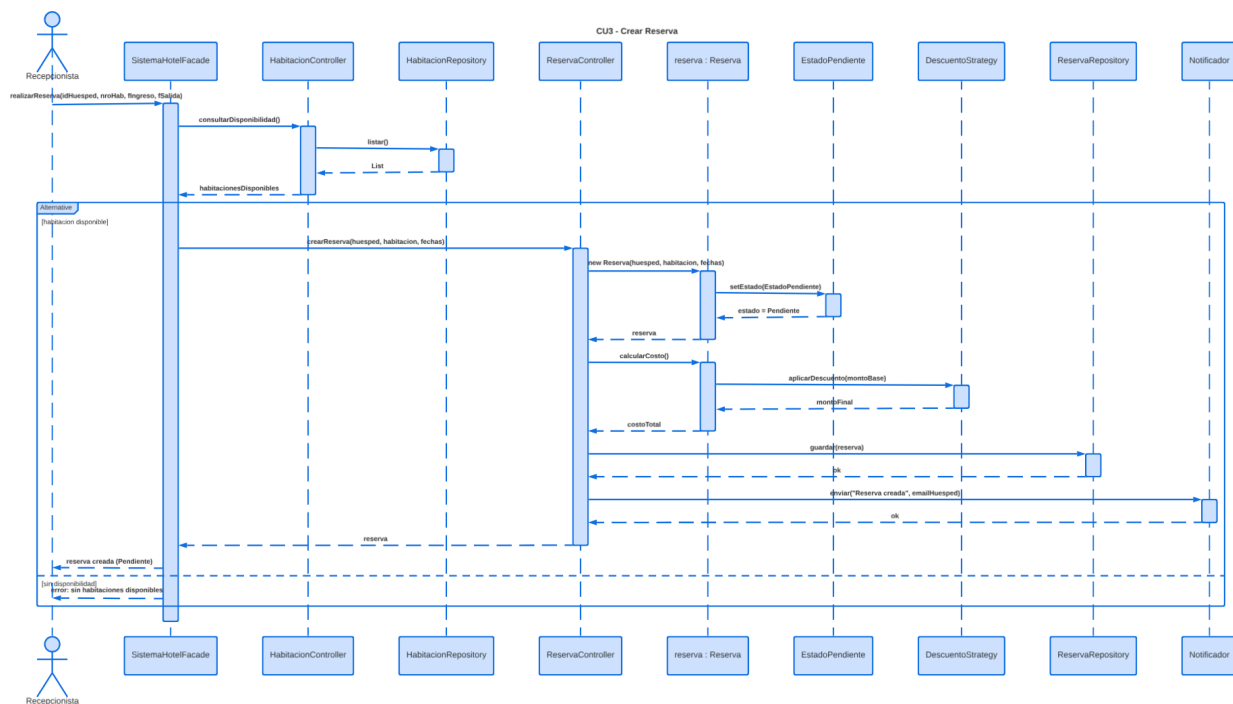


4. Diagramas de secuencia

Se presentan dos casos de uso relevantes que muestran la colaboración entre los objetos y la participación de los patrones de diseño en el tiempo de ejecución.

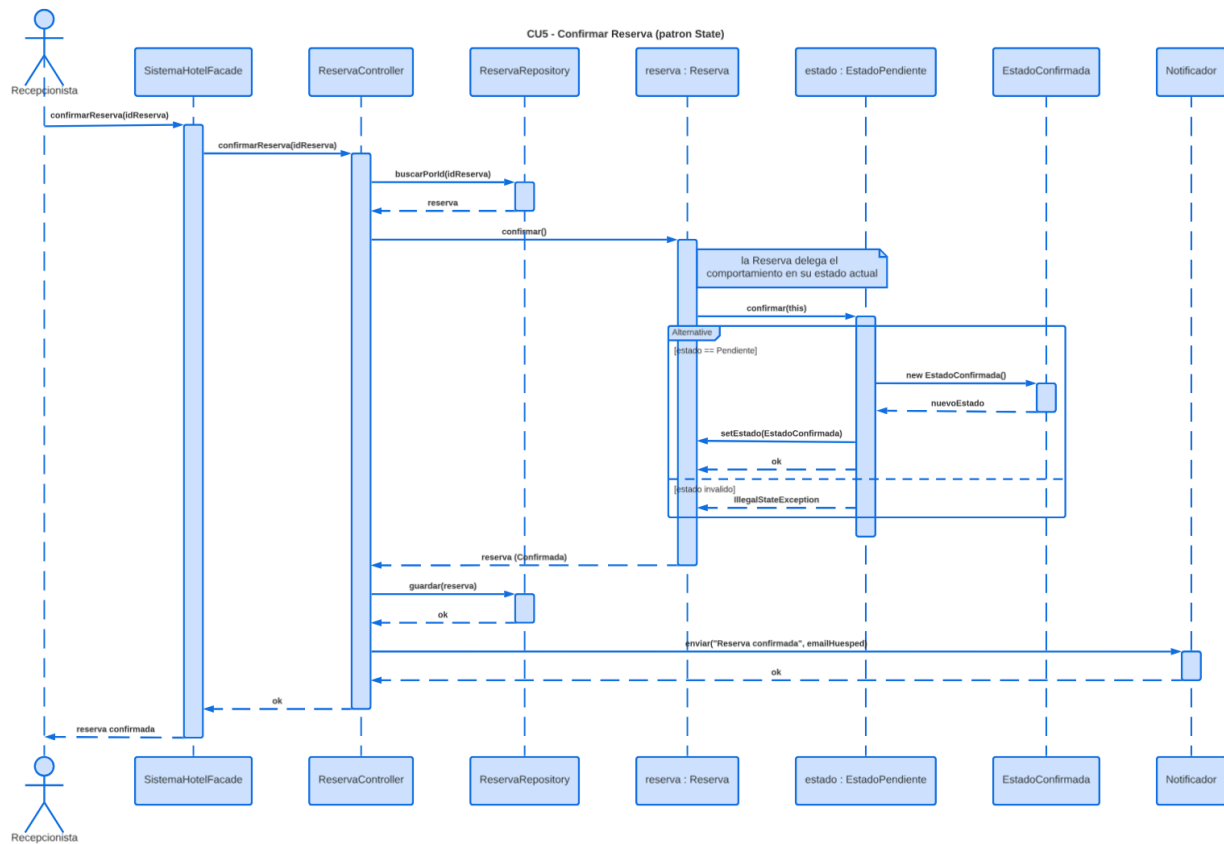
4.1. CU3 — Crear reserva

El recepcionista solicita una reserva a través de la fachada. La fachada consulta la disponibilidad mediante el controlador de habitaciones y, si existe una habitación libre, el controlador de reservas crea la Reserva (que nace en estado Pendiente), calcula su costo aplicando la estrategia de descuento, la persiste en el repositorio y notifica al huésped. El fragmento alternativo (alt) contempla el escenario sin disponibilidad.



4.2. CU5 — Confirmar reserva

El recepcionista confirma una reserva existente. El controlador recupera la reserva del repositorio e invoca confirmar(). La Reserva no decide por sí misma la transición: delega el comportamiento en su estado actual (patrón State). El estado Pendiente crea el estado Confirmada y actualiza la reserva; si el estado no admitiera la transición, se lanzaría una excepción controlada. Finalmente se persiste el cambio y se notifica al huésped.



5. Patrones de diseño aplicados y justificación

Cada patrón responde a una necesidad concreta del diseño. A continuación se justifica, para cada uno, el problema que resuelve dentro del sistema y la ventaja que aporta frente a una solución más acoplada.

5.1. Factory Method (creacional)

- Categoría: Patrón creacional — paquete factory
- Problema que resuelve: El sistema debe crear habitaciones de distintos tipos (Standard, Premium y Suite) sin acoplar el controlador a la lógica concreta de creación. Instanciar directamente las clases concretas dentro del controlador lo acoplaría a cada tipo y obligaría a modificarlo cada vez que se incorpore un nuevo tipo de habitación.
- Cómo se aplica: Se define la clase abstracta `HabitacionFactory` con el método `crearHabitacion()`, y las fábricas concretas `StandardFactory`, `PremiumFactory` y `SuiteFactory`. El `HabitacionController` selecciona la fábrica adecuada y delega en ella la creación.
- Ventaja frente a una solución rígida: Permite agregar nuevos tipos de habitación creando una nueva fábrica, sin modificar el código existente (respeta OCP). Centraliza y aísla la lógica de creación.

5.2. Singleton (creacional)

- Categoría: Patrón creacional — paquete repository
- Problema que resuelve: El sistema utiliza persistencia en base de datos. Mantener múltiples instancias de conexión sería ineficiente y podría provocar inconsistencias y consumo innecesario de recursos.
- Cómo se aplica: `DatabaseConnection` posee un constructor privado y un método estático `getInstancia()` que devuelve siempre la misma instancia. Todos los repositorios la utilizan para acceder a la base de datos.
- Ventaja frente a una solución rígida: Garantiza un único punto de acceso y configuración de la conexión durante toda la ejecución, evitando duplicación de recursos y facilitando el control centralizado de la persistencia.

5.3. Decorator (estructural)

- Categoría: Patrón estructural — paquete `model.servicio`
- Problema que resuelve: Una reserva puede incluir servicios adicionales (desayuno, spa, cochera) en cualquier combinación. Modelar cada combinación con herencia provocaría una explosión de subclases, y agregar atributos booleanos a la reserva la volvería rígida y difícil de extender.
- Cómo se aplica: Se define la interfaz `ServicioReserva`, el componente base `ReservaBase` y el decorador abstracto `ServicioDecorator`, del que heredan `ServicioDesayuno`,

ServicioSpa y ServicioCochera. Cada decorador envuelve al anterior y suma su descripción y costo.

- Ventaja frente a una solución rígida: Permite combinar servicios dinámicamente en tiempo de ejecución sin modificar la clase Reserva ni crear subclases por combinación. Nuevos servicios se agregan creando un nuevo decorador.

5.4. Facade (estructural)

- Categoría: Patrón estructural — paquete facade
- Problema que resuelve: El sistema tiene varios subsistemas (controladores de huéspedes, habitaciones, reservas y pagos, repositorios y notificadores). Que el cliente los coordine directamente generaría alto acoplamiento y duplicación de la lógica de orquestación.
- Cómo se aplica: SistemaHotelFacade expone operaciones de alto nivel (registrarHuesped, realizarReserva, confirmarReserva, etc.) e internamente coordina los controladores y la inicialización de la infraestructura.
- Ventaja frente a una solución rígida: Reduce el acoplamiento entre el cliente y los subsistemas (GRASP Bajo Acoplamiento), simplifica el uso del sistema y concentra la orquestación en un único punto.

5.5. State (comportamiento)

- Categoría: Patrón de comportamiento — paquete model.state
- Problema que resuelve: Una reserva atraviesa distintos estados (Pendiente, Confirmada, Cancelada, Finalizada) y no todas las transiciones son válidas (por ejemplo, no se puede finalizar una reserva pendiente). Resolverlo con condicionales (if/switch sobre el estado) dispersaría la lógica y la haría frágil ante nuevos estados.
- Cómo se aplica: Se define la interfaz EstadoReserva con los métodos confirmar(), cancelar() y finalizar(), y los estados concretos EstadoPendiente, EstadoConfirmada, EstadoCancelada y EstadoFinalizada. La Reserva delega cada operación en su estado actual, que decide la transición o lanza una excepción si es inválida.
- Ventaja frente a una solución rígida: Encapsula el comportamiento de cada estado en su propia clase, elimina los condicionales y facilita agregar nuevos estados. Las transiciones inválidas quedan controladas de forma explícita.

5.6. Strategy (comportamiento)

- Categoría: Patrón de comportamiento — paquete strategy
- Problema que resuelve: El cálculo del precio final puede aplicar distintas políticas de descuento (sin descuento, porcentaje fijo, temporada baja) y deben poder cambiarse o ampliarse sin alterar la lógica de la reserva.
- Cómo se aplica: Se define la interfaz DescuentoStrategy con el método aplicarDescuento(), y las estrategias concretas SinDescuento, DescuentoPorcentual y DescuentoTemporada. La Reserva recibe una estrategia y la utiliza al calcular su costo.

- Ventaja frente a una solución rígida: Permite intercambiar el algoritmo de descuento en tiempo de ejecución y agregar nuevas promociones creando una nueva estrategia, sin modificar la Reserva (respeto OCP).

6. Principios SOLID aplicados

Se aplican de forma concreta tres principios SOLID, justificados a nivel conceptual y vinculados al código del sistema.

Principio	Dónde se aplica	Justificación
SRP — Responsabilidad Única	Separación en capas: model, repository, controller, facade.	Cada clase tiene una única razón para cambiar. Los controladores coordinan casos de uso, los repositorios persisten, las clases de dominio modelan la información y la fachada orquesta. La lógica de negocio, la persistencia y la coordinación están separadas.
OCP — Abierto/Cerrado	Strategy (descuentos), Factory Method (habitaciones), State (estados).	El sistema está abierto a la extensión y cerrado a la modificación: se agregan nuevas promociones, tipos de habitación o estados creando nuevas clases, sin modificar las existentes.
DIP — Inversión de Dependencias	Notificador, Repository<T>, DescuentoStrategy, EstadoReserva.	Los módulos de alto nivel (controladores, reserva) dependen de abstracciones (interfaces) y no de implementaciones concretas. Por ejemplo, ReservaController depende de la interfaz Notificador, no de EmailNotificador.

7. Patrones GRASP aplicados

Se aplican cuatro patrones GRASP para lograr una correcta asignación de responsabilidades, bajo acoplamiento y alta cohesión.

Patrón GRASP	Dónde se aplica	Justificación
Experto en Información	Reserva.calcularCosto(), Habitacion.getPrecioPorNoche()	La responsabilidad se asigna a la clase que posee la información necesaria. La Reserva calcula su costo porque conoce

Patrón GRASP	Dónde se aplica	Justificación
		habitación, fechas, servicios y descuento; la Habitación conoce su precio.
Creador	Hotel crea/agrupa Reservas y Habitaciones; ReservaController instancia Reserva.	Se asigna la creación de objetos a quien los contiene o utiliza estrechamente, respetando las relaciones del dominio.
Controlador	ReservaController, HabitacionController, HuespedController, PagoController	Los casos de uso son coordinados por clases controladoras que reciben las solicitudes del actor y delegan en el dominio y la persistencia.
Bajo Acoplamiento	SistemaHotelFacade; uso de interfaces (Notificador, Repository).	La fachada y las interfaces minimizan las dependencias entre módulos, de modo que un cambio en un subsistema no impacte a los demás.

11. Informe de avance individual por integrante

De acuerdo con la distribución de tareas definida en la Fase 1, el avance de cada integrante durante esta fase fue el siguiente:

Tomás Szkarlatiuk

- Modelo de dominio:
 - Hotel
 - Huesped
 - Habitacion
 - Reserva
 - Promocion
 - Pago
- Implementación de los patrones:
 - State (EstadoReserva y sus estados concretos)
 - Decorator (ServicioDecorator y decoradores)
- Documentación del informe
- Commits realizados:
 - Implementación del modelo de dominio
 - Creación de clases restantes

Delia Tomás

- Capa de persistencia:
 - Repository<T>
 - ReservaRepository
 - HabitacionRepository
 - HuespedRepository
 - PagoRepository
- Implementación del patrón Singleton:
 - DatabaseConnection
- Diagrama de Secuencia CU5
- Aplicación de GRASP:
 - Bajo Acoplamiento (Low Coupling)
- Validación de compilación y ejecución del sistema
- Commit realizado:
 - Creación de repositories

Germán Rocha

- Implementación del patrón Factory Method:
 - HabitacionFactory
 - StandardFactory
 - PremiumFactory
 - SuiteFactory
- Implementación del patrón Facade:
 - SistemaHotelFacade
- Desarrollo de controladores:
 - ReservaController
 - HabitacionController
 - HuespedController
 - PagoController
- Análisis y documentación de principios:
 - SOLID
 - GRASP
- Diagramas realizados:
 - Diagrama de Clases UML
 - Diagrama de Secuencia CU3

- Gestión del repositorio Git
- Commits realizados:
 - o Creación de Facade
 - o Creación de Factory Method

Evidencia en Git: el trabajo se gestiona en el repositorio del grupo, donde cada integrante registra commits correspondientes a las tareas asignadas (implementación de clases, aplicación de patrones, documentación y diagramas).

- Repositorio GitHub del Proyecto
 - <https://github.com/131310100505/SistemaGestionHotelera>

12. Conclusión

En esta Fase 2 se concretó el diseño orientado a objetos del Sistema de Gestión Hotelera, avanzando coherentemente desde el análisis de la Fase 1 hacia un modelo de diseño y una primera versión de código funcional. Se definieron las responsabilidades de cada clase y se seleccionaron seis patrones de diseño que cubren las categorías creacional, estructural y de comportamiento, cada uno justificado por una necesidad concreta del sistema.

La arquitectura resultante presenta bajo acoplamiento y alta cohesión, con una clara separación de capas (dominio, persistencia, controladores y fachada) y la aplicación de los principios SOLID (SRP, OCP, DIP) y patrones GRASP (Experto, Creador, Controlador, Bajo Acoplamiento). El código se compila y se ejecuta correctamente, demostrando los casos de uso principales y la colaboración de los patrones en el tiempo de ejecución.

El diseño queda preparado para la Fase 3, en la que se completará la implementación, se incorporará la persistencia real mediante base de datos relacional y se ampliará la batería de pruebas, manteniendo la coherencia entre análisis, diseño, patrones, código y pruebas.